Exercise 6 – Shapley Values

Interpretable Machine Learning

Hint: Useful libraries

R

```r
library(sets)       # power sets (Exercise 1)
library(combinat)   # permutations (Exercise 1)
library(ranger)     # random forest (Exercise 3)
library(randomForest)  # random forest for the LIME explainer (Exercise 4)
library(DALEX)      # model explainers (Exercise 4)
library(DALEXtra)   # LIME interface (Exercise 4)

set.seed(123456)
```

Python

```python
import itertools
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.special import comb
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import lime
import lime.lime_tabular

np.random.seed(123456)
```

Exercise 1: The Shapley value

Learning goals

1. Compute payoffs of coalitions in a cooperative game.
2. Implement the exact Shapley value via both the set and the permutation definition.
3. Implement the permutation-based approximation of Shapley values.
4. Check the Shapley axioms empirically for a concrete game.

In this exercise, we implement the original Shapley value for a cooperative game. The game we consider here is an AI quiz, where we have five players: Timnit, Margret, Samy, Jeff, and Larry. The overall payoff function for a set of players S is given as

$$\text{payoff}(S) = 10t + 10m + 10s + 2j + 20(t \wedge m) + 20(t \wedge m \wedge s) - 30((t \vee m \vee s) \wedge j)$$

where t, m, s, j, l indicate whether Timnit, Margret, Samy, Jeff, and Larry are in the set S. The function `payoff(coalition)` is supposed to implement this functionality, where a set is a list/set of the respective characters (`'t', 's', 'm', 'j', 'l'`).

a) Calculate the payoff of $S = \{$`t,m`$\}$ (result should be 40) and $S = \{$`t,j,s`$\}$ (result should be -8).

b) Implement the original, exact Shapley value algorithm, using the set definition of Shapley values:

- Start by implementing the function `payoff(coalition)`, which should return the payoff of a subset `coalition` as given in the task. You can do this, for example, by first assigning a Boolean value for each player.
- Define a function `all_unique_subsets(population)` returning all subsets of a given set `population`.
- Finally, implement `shapley_set(member, population, v_function)` calculating the Shapley value for an arbitrary value function using the set definition.

Test your functions with the examples from a) or with other examples from this game, i.p. corner cases like the empty set or the full set of all players.

c) Analogously, implement the exact Shapley value algorithm using the order definition (or permutation definition) `shapley_perm(member, population, v_function)`. For this purpose, you can write (or use) a function that returns all orderings / all permutations of a given set. Again, test your function as in b).

d) Next, implement the permutation-based approximation discussed in the lecture: `shapley_perm_approx(member, population, v_function, num_iter)`. Here, `num_iter` is a fixed number of iterations given.

Note that because we implement the Shapley values for game theory, not yet for machine learning, we are not concerned with PD-functions and with data sets, but we can calculate the payoff function directly. This means that sampling is only required concerning the permutations, the sampling concerning the data points can be omitted.

Again, don't forget to test your function. Compare it to the function in c) concerning runtime and accuracy for different numbers of iterations.

e) In the last part of the exercise, we want to test whether the Shapley values or specific sets fulfill the axioms that uniquely define the Shapley values. Please write the functions `efficiency_check`, `symmetry_check`, `dummy_check`, and `additivity_check` to check the respective properties, all of which return a Boolean value. We briefly recall the properties below:

(i) *Efficiency:* Player contributions add up to the total payout of the game:

$$\sum_{j=1}^{p} \phi_j = v(P).$$

(ii) *Symmetry:* Assess whether two features with the same contributions have the same Shapley value. I.e., for any two players $\{j, k\}$, if for any $S \subseteq P \setminus \{j, k\}$ we have

$$v(S \cup \{j\}) - v(S) = v(S \cup \{k\}) - v(S),$$

then the Shapley values of j and k should be identical.

(iii) *Dummy:* Given a player j which makes no contribution for all subsets $S \subseteq P \setminus \{j\}$ (that is $v(S) = v(S \cup \{j\})$), the Shapley value of this player must be zero.

(iv) *Additivity:* If v is the sum of two games v_1 and v_2 (i.e. $v(S) = v_1(S) + v_2(S)$ for any S), then the payout is the sum of payouts: $\phi_{j,v} = \phi_{j,v_1} + \phi_{j,v_2}$.

Again, test these functions and use them to check whether the axioms are fulfilled for our specific example.

Exercise 2: Axioms of Shapley Values

Learning goals

1. Prove the symmetry, dummy and additivity axioms of Shapley values.
2. Explain the relationship between the set and the permutation definition of Shapley values.

3. Apply proof techniques such as index transformations and telescope sums to the Shapley formula.

In this exercise, we want to mathematically prove that the Shapley values in cooperative game theory, as defined in the lecture, fulfill the four axioms discussed in the lecture. Throughout this exercise, we assume that the order and the set definition of Shapley values yield the same result, so you can always choose which definition to use.

a) Prove that the symmetry, dummy player, and additivity axioms of Shapley values hold for any value function and any set P of players. These axioms are rather straightforward to prove, and this can be done using either of the definitions.

b) **Bonus:** Prove the efficiency axiom for Shapley values.

Hint

This is easier when using the order definition. Plug the definition into the sum inside the axiom, then swap the sums and argue why almost all terms of the inner sum vanish.

c) **Bonus:** Similarly to the additivity axiom, prove the following homogeneity axiom:

If $\alpha \in \mathbb{R}$ is a real number and v the payout function of a game, and we define $\tilde{v} := \alpha v$ as another payout function, then the Shapley value also scales accordingly, that is for all features $j \in P$ we have:

$$\phi_{j,\tilde{v}} = \phi_{j,\alpha v} = \alpha \phi_{j,v}.$$

Note: This axiom together with the additivity axiom shows that the Shapley value is linear in the payout function, so we have linearity of the Shapley value.

Exercise 3: SHAP

Learning goals

1. Apply Shapley values as a local feature relevance tool to a machine learning model.
2. Implement the marginal sampling-based SHAP value function and plug it into the Shapley approximation.
3. Implement Kernel SHAP and interpret the resulting SHAP values.

In this exercise, we want to apply Shapley values as a local feature relevance quantification tool to a machine learning model and also use the SHAP framework for this. We will implement both the simple sampling-based approximation algorithm for vanilla Shapley values from the lecture, as well as the Kernel SHAP algorithm.

a) Load the FIFA dataset (containing all matches from the FIFA world cup 2018) from data/fifa.csv into a data table or data frame (e.g., using the pandas library in Python) and train a random forest to predict the target variable 'Man of the Match'. This means that the model predicts the probability of a team having the 'Man of the Match' amongst them.

Hint

Note that many of the float variables in the data set contain missing values, hence you may consider to use integer variables only. Transform the target variable 'Man of the Match' into a binary format suitable for a prediction task and select only explanatory variables of type integer, or only those without missing values. Don't forget to split the data into a training and test set before fitting the random forest! Use an instance from your test set to generate an exemplary prediction for the 'Man of the Match' probability.

b) From the different possibilities to choose the value function $g(\mathbf{z}')$ in the SHAP framework, here we use the marginal sampling-based SHAP value function $g(\mathbf{z}') = \mathbb{E}_{\mathbf{X}_{-S}}[\hat{f}(\mathbf{x}_S, \mathbf{X}_{-S})]$, so we use PD-functions. Implement the function `marginal_vfunc(S, observation, X, predict, nr_samples)`, which approximates the PD-function value for a feature subset `S`, evaluated at the data point `observation`, via sampling background data from the data set `X`; in other words, implement a Monte-Carlo approximation of this PD-function evaluated at point `observation`. `predict` is a function that returns the model predictions for a single data point or a set of data points.

Test your implementation on different feature subsets for the instance from a).

c) By using this value function `marginal_vfunc()` inside our Shapley value implementations from Exercise 1, we can approximate the Shapley values for our model. This is almost the approximation algorithm for Shapley values discussed in the lecture.

Calculate `shapley_perm_approx()` from the Shapley value implementations using `marginal_vfunc()` and the random forest from a) on your test instance.

d) The SHAP framework, which is based on locally fitting a weighted linear model, can be used to calculate SHAP values. Besides the vanilla framework, which basically uses the same algorithm as the one we implemented so far, other more efficient equivalent algorithms for calculating SHAP values have been proposed, among them Kernel SHAP. Your task is to implement Kernel SHAP and calculate the SHAP values for your test instance. Do this by writing the following functions:

- `shap_weights(mask)`: A function to return the weights for a given `mask` containing the sampled coalitions, so `mask` is either a single coalition vector or a matrix containing K many coalition vectors.

- `replace_dataset(obs, X, nr_samples)`: A function to create the dataset of `nr_samples` many samples generated in the first and second step of the Kernel SHAP calculations. This includes generating the coalitions, saving them in `mask`, and mapping the binary feature space back to the original feature space. You can directly sample `nr_samples` many data points as well as coalitions, so sampling a new random background sample for each coalition, you do not need to generate multiple background samples for every coalition or use the same background sample for different coalitions. The positions in the mask indicate where to replace the random values with the ones of the given observation `obs`.
- `shap_data(obs, X, nr_samples, predict)`: A function summarizing `replace_dataset()`, `predict()`, and `shap_weights()`. As before, `predict` is a function that returns the model predictions for a single data point or a set of data points.
- `kernel_shap(obs, X, nr_samples, predict)`: A function implementing the complete sampling-based Kernel SHAP function.

e) Calculate `kernel_shap(X_test[1, ], X_train, 1000, classifier_RF)` using the random forest from a) and interpret the resulting SHAP values in this context.

Exercise 4: LIME

Learning goals

1. Apply an existing LIME implementation to a fitted model.
2. Explain the influence of hyperparameters such as the kernel width on LIME explanations.
3. Compare LIME and SHAP conceptually.

LIME is a local feature effect method that is based on locally approximating the decision function using a linear model. Then, the linear model coefficients are interpreted as usual. Like SHAP, LIME is an additive attribution technique.

a) Apply a LIME implementation to the dataset from Exercise 3.

Hint

Implementations are available in Python and R.

b) Try different hyperparameter configurations (for instance, modify the kernel width). How does that influence the result?

c) Compare the result to SHAP. In what way are LIME and SHAP similar? How do the methods differ?